MODULE 36 · DOMAIN 4 - OPERATING, MONITORING, SECURING

Guardrails and responsible AI

Six safeguard policies, ApplyGuardrail, and securing the pipeline itself.

MODULE 36 of 43 DOMAIN 4 - Monitoring WEIGHT 24%

admin@sthithapragna.com

An instruction asks a model to behave. A guardrail does not ask.

Guardrail safeguard policies

POLICY	WHAT IT DOES
Content filters	Blocks harmful content, and prompt attacks. Configurable strength
Word filters	Exact-match words and phrases, plus a managed profanity list
Denied topics	Topics defined in natural language, blocked in input or output
Sensitive information filters	PII entities and custom regex. Block or mask
Contextual grounding checks	Scores a response against a source and the query. Probabilistic
Automated Reasoning checks	Formal logic verification against an authored policy

ON THE EXAM

The last two both address hallucination and they work on completely different principles. That contrast is the most testable thing here.

Content filters

- Categories cover **hate, insults, sexual content, violence and misconduct**.
- A separate **prompt attack** filter addresses jailbreaks and prompt injection. Module 16.
- Each has a configurable strength: **NONE, LOW, MEDIUM or HIGH**.
- They apply to **both input and output**, which are different problems.
- Higher strength means more false positives. Module 17's precision and recall, in a safety setting.

> ON THE EXAM
>
> The strength setting is a threshold, so module 17 applies exactly: a stricter filter blocks more harm and more legitimate requests.

Choosing filter strength

- Start at **MEDIUM** and move based on evidence rather than instinct.
- **Raise it** where the consequence of harmful output is severe: public-facing, minors, regulated advice.
- **Lower it** where legitimate content resembles a blocked category: medical, legal, security research.
- A security team asking about attack techniques trips a filter that a retail bot should keep at HIGH.
- Different applications warrant different guardrails. One organisation-wide setting rarely fits all.

ON THE EXAM

Module 17's cost-of-errors question again: what does a false block cost here, against what a missed harmful output costs?

Word filters and denied topics

- **Word filters** are exact-match on words and phrases, plus a managed profanity list.
- Useful for more than profanity: **competitor names**, product names, internal codenames.
- **Denied topics** are defined in **natural language**, so they generalise beyond exact wording.
- An HR assistant can deny legal advice; a retail bot can deny medical questions.
- A topic definition that is too broad blocks legitimate questions. Test with real ones.

> ON THE EXAM
>
> Word filters catch a string. Denied topics catch an intent. A user rephrasing defeats the first and not the second.

Sensitive information filters

- Detects **PII entities**, and supports **custom regex** for your own identifiers.
- Two actions: **BLOCK** the request entirely, or **MASK** the entity and continue.
- **Block** what must never appear, such as credentials or access keys.
- **Mask** what should be scrubbed while keeping the text usable: names, emails, phone numbers.
- Mixing both in one policy is normal, and usually correct.

> ON THE EXAM
>
> Masking is the one people underuse. A summarisation workflow does not need the customer's name, and blocking would break it entirely.

Images and multimodal content

- Guardrail evaluation extends beyond text to **image content** where supported.
- That matters because a multimodal application accepts inputs your text filters never see.
- Module 10's multimodal documents and module 26's image insights both produce this exposure.
- An input channel with no safeguard is the one an attacker will find.
- Check current support before designing around it. Coverage here is expanding.

> ON THE EXAM
>
> Every input channel needs a policy. A text-only guardrail on a multimodal application covers the easy half.

Contextual grounding checks

- Scores a response against a **grounding source** you supply, and against the **query**.
- **Grounding** asks: is this supported by the source? **Relevance** asks: does it answer the question?
- That is module 18's **faithfulness**, enforced at run time rather than measured in evaluation.
- The natural fit is a RAG application, where the retrieved chunks are the grounding source.
- It is **probabilistic scoring** with a threshold, so it can be wrong in both directions.

> ON THE EXAM
>
> Module 18 measured faithfulness after the fact on an evaluation set. This blocks an unfaithful answer before the user sees it.

Automated Reasoning checks

- Verifies claims using **formal logic**, against a structured policy of known facts you author.
- AWS calls it **the first and only generative AI safeguard to use formal logic**.
- It validates correct responses with **up to 99 percent accuracy**, and can **explain** its finding.
- It requires authoring a policy, which is real work and the reason people hesitate.
- Right where rules are genuinely formal: eligibility, entitlements, documented policy.

> ON THE EXAM
>
> This is the only safeguard here that can prove a claim wrong rather than score it as unlikely. That is a different kind of guarantee.

Contextual grounding or Automated Reasoning?

CONTEXTUAL GROUNDING

Probabilistic, against a source

- You supply the source at request time.
- Scores support and relevance.
- No authoring required.
- Natural fit for RAG.

AUTOMATED REASONING

Formal logic, against a policy

- You author a policy of known facts once.
- Verifies claims logically.
- Can explain why a claim is wrong.
- Natural fit for rules and entitlements.

THE TIEBREAKER

Grounding needs a source per request. Automated Reasoning needs a policy authored once. That is usually what decides it.

Classic and Standard

- Content filters and denied topics come in a **Classic** and a **Standard** tier.
- **Standard** handles modifications such as typographical errors, and supports **up to 60 languages**.
- Its prompt attack filter **distinguishes jailbreaks from prompt injection**, and resists output manipulation.
- **Standard requires explicitly opting in to cross-Region inference** with Guardrails.
- So module 08's residency constraint can prevent you using the stronger tier.

> ON THE EXAM
>
> The residency tension again: the better safeguard requires cross-Region inference, which some organisations cannot permit.

The ApplyGuardrail API

- Evaluates content **independently of the model that produced it**.
- So one guardrail protects **Bedrock models, self-hosted models on SageMaker AI, and third-party APIs**.
- You set **source** to INPUT or OUTPUT, so the guardrail knows which it is evaluating.
- That gives one safety layer across a mixed portfolio, with no vendor lock-in on the safety layer.
- The application must actually call it, which is the weakness the next slide addresses.

> ON THE EXAM
>
> This is the answer whenever a scenario mentions a model outside Bedrock and a requirement for consistent safety policy.

Content blocks and qualifiers

- **content is an array of blocks**, and qualifiers say what each block is for.
- **guard_content** is the text to check. **grounding_source** is the source. **query** is the question.
- **Batching response, source and query into one call** is how contextual grounding is meant to be used.
- It is also cheaper in call volume than evaluating each piece separately. Module 33.
- Evaluating the response alone gives grounding nothing to compare against.

> ON THE EXAM
>
> The qualifiers are not decoration. Without a grounding source, the grounding check has no source, and it will not do the job you enabled it for.

IAM policy-based enforcement

- A guardrail the application forgets to attach protects nothing at all.
- **IAM policy conditions can require that a specific guardrail is applied** to inference calls.
- So an invocation without the guardrail is **denied**, rather than proceeding unprotected.
- That turns a code review item into an account-level control. Module 34's pattern exactly.
- Combine with an SCP to apply it across every account in the organisation.

> ON THE EXAM
>
> This is the answer whenever a scenario says developers keep forgetting, or a new service was deployed without the safety layer.

Central enforcement

- A central security team usually wants **one policy applied everywhere**, not nine copies.
- IAM enforcement plus an **SCP** applies it across every account. Module 34.
- AWS has been extending Guardrails toward **cross-account enforcement**, so check what is available.
- Central control and local tuning pull against each other, exactly as with filter strength.
- A workable shape: a mandatory organisation baseline, plus stricter local policies where warranted.

ON THE EXAM

Baseline centrally, tighten locally. A single global setting either blocks a legitimate team or under-protects a sensitive one.

Blocked messages and observability

- **A blocked prompt returns a message you write**, not an AWS default.
- So the refusal can be helpful: explain what the assistant does and does not cover.
- A blunt refusal on a reasonable question is module 18's **answer refusal** failure.
- Masked content continues to the model with placeholders, so the request still works.
- **InvocationsIntervened** tells you how often this is happening, by policy and by source. Module 30.

> ON THE EXAM
>
> The blocked message is part of the product, not an error page. Most teams leave it as a sentence somebody typed once.

Guardrail economics

- Guardrails are charged **per assessment**, and checking input and output means two per request.
- **Contextual grounding and Automated Reasoning are dearer** than the simpler policies.
- So enabling every policy on every request is a real cost decision, not a free default.
- Batching content blocks into one ApplyGuardrail call is cheaper than several calls. Module 33.
- It is still cheap against the cost of one harmful output reaching a customer.

> ON THE EXAM
>
> Worth costing deliberately, and worth ending the conversation quickly: the asymmetry here is not close.

Guardrails for code

- Coverage now extends to **harmful content within code elements**, not just prose.
- It can detect **malicious code injection attempts**.
- And it prevents **PII exposure in code structures**: variable names, comments, test fixtures.
- That matters for coding assistants and for any agent that writes or reads code.
- Module 27's Code Interpreter is exactly the case this protects.

> ON THE EXAM
>
> PII hidden in a comment or a test fixture is invisible to a prose-oriented filter, and it is a common way real data leaks into code.

Guardrail versions and testing

- A guardrail is **versioned**, and an application references a specific version.
- So tightening a policy is a **deployment**, not an edit that takes effect everywhere at once.
- Test in the console against real prompts, including the ones you expect to be borderline.
- **Test the false positives**: legitimate questions that a stricter policy might now refuse.
- Working drafts let you iterate before publishing a version anyone depends on.

> ON THE EXAM
>
> A policy change is a production change. Module 16 said that about prompts, and it is equally true here.

Guardrails across the stack

- On a **model invocation**, through Converse or InvokeModel.
- On a **Knowledge Base** RetrieveAndGenerate call, from module 26.
- On an **agent**, where it matters mid-loop because tool results become model input. Module 27.
- And on **anything at all**, through ApplyGuardrail called directly.
- An agent with a guardrail only on the final answer leaves every intermediate step unguarded.

> ON THE EXAM
>
> The agent case is the one to think about: a prompt-injected tool result is model input, and it arrives after your input check has run.

Responsible AI is not a policy document. It is a set of **engineering decisions** you already know how to make.

Responsible AI, and where this course addressed it

DIMENSION	WHERE IT WAS BUILT
Fairness	Bias metrics before and after training. Modules 08 and 19
Explainability	SHAP, partial dependence, online explainability. Module 19
Privacy and security	Encryption, isolation, PII filters. Modules 08, 34, 35
Safety	Guardrails, and the refusal behaviour around them
Robustness	Evaluation, drift monitoring, rollback. Modules 17, 24, 29
Transparency	Model cards, citations, lineage. Modules 19, 26
Governance	Registry approval, audit trail, policy enforcement. Modules 21, 31, 34

ON THE EXAM

Seven dimensions, and none of them is new work. The course has been building this quietly since module 08.

Bias in generated output

- Module 08's bias metrics need labels and groups. **Generated text has neither.**
- So the measurement is different: **stereotyping** as an evaluation metric, from module 18.
- Probe deliberately: the same question with different names, genders or locations, and compare answers.
- Guardrails can block stereotyping content. Evaluation tells you how often it would have occurred.
- A retrieval system inherits the bias of its corpus, which no filter on the output will fix.

> ON THE EXAM
>
> The last point is the uncomfortable one. If the documents are one-sided, faithful retrieval reproduces that faithfully.

Documenting a generative system

WHAT TO RECORD	WHY IT IS ASKED FOR
Intended use, and out-of-scope use	The first question in any review
Which model and version, and why	Model choice is a decision with consequences. Module 16
Guardrail configuration and its version	What is blocked, and since when
Evaluation results and the set used	Evidence of quality. Modules 18 and 30
Data sources and their permissions	What the system can see. Module 26
Human oversight points	Where a person is in the loop, and where they are not

ON THE EXAM

This is a model card for a generative system. Module 19 built the artefact; these are the rows that differ.

Responding to an AI incident

- **Contain first:** tighten the guardrail, disable the feature, or route to a human.
- **Then investigate with the trace**, from module 30. A bad answer is a sequence, not a string.
- Invocation logs give the exact prompt and response, if they were enabled. Module 30.
- Add the case to the evaluation set, so the fix is verified and stays verified.
- Model, prompt, guardrail and documents can all change without a deployment. Record which did.

> ON THE EXAM
>
> Containment is faster here than in classical ML: tightening a guardrail or raising a threshold needs no redeployment.

Human oversight

- **Human review** on low-confidence predictions, from module 07.
- **Human approval** before an agent takes an irreversible action, from module 27.
- **Model approval** before deployment, as an IAM-controlled step. Modules 21 and 34.
- **A route to a person** when the system cannot help, which is a product decision more than a technical one.
- Oversight that nobody has time to perform is oversight in name only. Reserve it for what warrants it.

> ON THE EXAM
>
> Four different human checkpoints, at four different stages. A system with none of them is automated in a way somebody will eventually regret.

Red teaming a generative system

- Deliberately try to make the system misbehave, before somebody else does it for free.
- Test **jailbreaks, injection through retrieved content, and PII extraction**. Modules 16 and 26.
- Test the boring failures too: over-refusal, and confidently wrong answers on in-scope questions.
- Turn every finding into a case in the evaluation set, from module 18.
- A system nobody has attacked has not been shown to be safe. It has been shown to be untested.

> ON THE EXAM
>
> Module 24's rollback, module 28's gate, module 35's isolation, and now this: the control you have never seen fire is the one that will not.

You can secure the model perfectly and ship it through a **pipeline nobody checked**.

Amazon Inspector

- Scans **container images in ECR** and compute for known vulnerabilities, continuously.
- Your training and inference images are exactly that: containers, with dependencies, that age.
- A base image pinned for reproducibility in module 21 is a base image that stops receiving patches.
- Findings can gate a pipeline, so a vulnerable image does not reach production. Module 28.
- That creates a real tension: reproducibility wants pinning, and security wants updating.

> ON THE EXAM
>
> This tension is genuine and worth naming. The resolution is usually a scheduled rebuild, so pinning is deliberate rather than permanent.

Code and secret scanning in the pipeline

- Static analysis catches defects and security issues in training and inference code.
- **Secret scanning** is the highest-value check: a committed credential is compromised immediately.
- Notebooks are code too, and they are where credentials and connection strings get pasted.
- Dependency scanning covers the libraries your model actually imports.
- Run these as pipeline stages, so they gate rather than advise. Module 28.

> ON THE EXAM
>
> Notebooks are the blind spot: they are rarely reviewed like application code, and they are where exploratory shortcuts live.

The ML supply chain

- **A pretrained model is executable content from the internet.** Treat it accordingly.
- Prefer models from **JumpStart or Bedrock** over an arbitrary download from a public hub.
- **Pin versions** for models, images and packages, so what runs is what you reviewed.
- **Network isolation stops a model artefact calling out** at load time. Module 35.
- A private registry keeps image pulls inside your VPC. Also module 35.

> ON THE EXAM
>
> Some serialisation formats execute code on load. 'Just download the weights' is not a neutral act.

Regulation, briefly

- Several jurisdictions now regulate A I systems by **risk level** and by **use case**.
- Common requirements: **transparency, human oversight, documentation, and the ability to explain**.
- Every one of those maps onto something this course already built.
- Specifics vary by jurisdiction and change, so this is a pointer rather than a summary.
- The engineering answer is the same regardless: build the evidence as you go.

> ON THE EXAM
>
> Retrofitting explainability, lineage and evaluation onto a finished system is far harder than building them in.

Requirement to control

WHAT THE SCENARIO ASKS FOR	THE CONTROL
Block harmful content and jailbreak attempts	Content filters, including the prompt attack filter
Keep the assistant off certain subjects	Denied topics, defined in natural language
Redact personal data but keep the request usable	Sensitive information filters, MASK action
Stop answers not supported by the retrieved sources	Contextual grounding checks
Verify claims against documented policy rules	Automated Reasoning checks
Same policy across Bedrock and a self-hosted model	The ApplyGuardrail API
Developers keep forgetting to attach the guardrail	IAM policy-based enforcement
A vulnerable container image reached production	Inspector, gating the pipeline

ON THE EXAM

Rows four and five are the pair to be careful with: probabilistic against a supplied source, or formal logic against an authored policy.

The safety setup to ship with

DECISION	WHY IT BELONGS ON DAY ONE
A guardrail, attached and IAM-enforced	An unenforced guardrail is a suggestion
A blocked message you actually wrote	The default is a sentence nobody designed
Contextual grounding on any RAG path	Unsupported answers are the characteristic failure
Invocation logging, with masking	Module 30: it cannot be enabled retrospectively
An evaluation set including refusal cases	Over-refusal is invisible in operational metrics
A documented out-of-scope list	The first thing any reviewer asks for

ON THE EXAM

Two of these six cannot be added retrospectively, and the other four get harder the longer a system has been live.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Guardrail exists, unsafe output still reached users	The application did not attach it. Enforce with IAM
Grounding check never intervenes	No grounding_source supplied. It has nothing to compare
Legitimate questions refused	Filter strength too high, or a denied topic too broad
A misspelled jailbreak got through	Classic tier. Standard handles modifications
Cannot enable Standard tier	It requires opting in to cross-Region inference. Module 08
PII appeared in generated code	Prose-oriented filtering only. Code coverage exists
Vulnerable base image in production for months	Pinned for reproducibility and never rebuilt

ON THE EXAM

Row two is the quiet one. A guardrail that never intervenes looks like a clean system, and is sometimes a misconfigured one.

A safeguard the application can forget to call is a suggestion.

A company runs three generative applications: one on Bedrock, one on an open model hosted on a SageMaker AI endpoint, and one calling a third-party API directly.

Compliance requires the same content safety and PII policy across all three, with a single place to change it.

How should they implement it?

A Configure a Bedrock guardrail, and replicate the rules in code for the other two

B Move all three applications onto Bedrock so one guardrail applies

C Define one guardrail and call the ApplyGuardrail API from all three applications

D Use sensitive information filters on Bedrock, and Macie on the other two

◆ PAUSE AND DECIDE

How should they implement it?

A Configure a Bedrock guardrail, and replicate the rules in code for the other two

Three implementations that will diverge, which is module 04's skew problem in a safety context. It also fails 'a single place to change it'.

B Move all three applications onto Bedrock so one guardrail applies

A large migration to obtain something the ApplyGuardrail API already provides without moving anything.

C Define one guardrail and call the ApplyGuardrail API from all three applications

It evaluates content independently of the model, covering Bedrock, self-hosted and third-party alike. One configuration, one place to change it.

D Use sensitive information filters on Bedrock, and Macie on the other two

Macie classifies data at rest in S3. It does not evaluate a prompt or a response in flight.

THE SIGNAL

'A model outside Bedrock' plus 'one consistent safety policy' is the ApplyGuardrail signal, and it appears in scenarios almost verbatim.

> A RAG assistant over an insurance policy library occasionally states excess amounts and exclusions that do not appear in any retrieved document. Retrieval quality is good and the correct passages are being returned.
>
> The team wants to stop unsupported statements reaching the user, at request time.

What should they enable?

A Contextual grounding checks, supplying the retrieved chunks as the grounding source

B Denied topics covering excess amounts and exclusions

C Content filters at HIGH strength on the output

D Scheduled Bedrock RAG evaluation on faithfulness

◆ PAUSE AND DECIDE

What should they enable?

A Contextual grounding checks, supplying the retrieved chunks as the grounding source

It scores the response against the source at request time and blocks unsupported statements before the user sees them. Exactly the described failure.

B Denied topics covering excess amounts and exclusions

Would block the assistant from discussing the very subjects it exists to answer. It blocks the topic, not the unsupported claim.

C Content filters at HIGH strength on the output

Content filters address harmful content. A fabricated excess amount is wrong, not harmful, and passes every category.

D Scheduled Bedrock RAG evaluation on faithfulness

Measures the problem across an evaluation set, which the team should also do. It does not stop a specific answer at request time.

THE SIGNAL

Note the difference from module 18: evaluation measures faithfulness afterwards, and a grounding check enforces it in the request.

> A security review finds that two of nine internal generative applications call Bedrock without attaching the organisation's approved guardrail. Both were built by teams who were unaware of it.
>
> Leadership asks for a control that does not depend on every team remembering.

What should they implement?

A. A written standard, with guardrail attachment added to the code review checklist

B. A CloudWatch alarm on invocations where InvocationsIntervened is zero

C. Content filters at HIGH strength on the guardrail, to compensate

D. IAM policy-based enforcement requiring the guardrail on inference calls

◆ PAUSE AND DECIDE

What should they implement?

A A written standard, with guardrail attachment added to the code review checklist

Exactly what the request rules out. A checklist depends on every team remembering, which is how the two applications happened.

B A CloudWatch alarm on invocations where InvocationsIntervened is zero

Detects after the fact, and a well-behaved application legitimately intervenes rarely. It would alarm constantly on healthy traffic.

C Content filters at HIGH strength on the guardrail, to compensate

Strengthens a guardrail those two applications never call. The problem is attachment, not strength.

D IAM policy-based enforcement requiring the guardrail on inference calls

An invocation without the guardrail is denied outright, so nothing reaches a model unprotected regardless of what a team knew.

THE SIGNAL

'Does not depend on every team remembering' is the enforcement signal. The same phrasing pointed at SCPs in module 34.

> A team wants the assistant's responses to always use the company's formal tone and never use contractions or exclamation marks. An engineer proposes configuring Automated Reasoning checks to enforce this.
>
> They have already authored a policy document describing the tone rules.

How should you respond?

A Agree. A written policy document is exactly what Automated Reasoning consumes

B Redirect. Automated Reasoning verifies factual claims with formal logic. Tone is a prompt and evaluation problem

C Redirect to word filters, blocking every contraction and exclamation mark

D Agree, but use contextual grounding instead, with the policy as the grounding source

◆ PAUSE AND DECIDE

How should you respond?

A Agree. A written policy document is exactly what Automated Reasoning consumes

It consumes a structured policy of logically verifiable facts. A tone guide is a style preference, not a set of claims that can be true or false.

B Redirect. Automated Reasoning verifies factual claims with formal logic. Tone is a prompt and evaluation problem

Tone is set by the prompt, from module 16, and measured by an LLM-as-a-judge style metric, from module 18. Neither is a logical verification.

C Redirect to word filters, blocking every contraction and exclamation mark

Technically possible and a poor design: it blocks the whole response over a punctuation mark rather than improving the writing.

D Agree, but use contextual grounding instead, with the policy as the grounding source

Grounding checks whether a response is supported by a source. A tone guide does not support or contradict the content of an answer.

THE SIGNAL

Module 18 already had a metric for this: 'professional style and tone' is an LLM-as-a-judge criterion, not a safeguard.

What to take from this module

- **Six policies**, and the two hallucination ones differ: probabilistic grounding, formal-logic reasoning.
- **ApplyGuardrail works with any model**, including self-hosted and third-party.
- **IAM policy-based enforcement** is how a guardrail stops being optional.
- **Standard tier requires cross-Region inference**, which residency may forbid.
- **Secure the pipeline too.** Scan images, scan code, and pin the supply chain.

> ON THE EXAM
>
> That closes the teaching content. Modules 37 to 40 are exam skills, and 41 to 43 are practice.